

Rückblick - Was haben wir letztes Mal gemacht?

Was stimmt hier nicht?

```
void setup() {  
  pinMode(13, OUTPUT);    // Pin als Ausgang setzen  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000)  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Rückblick - Was haben wir letztes Mal gemacht?

Was stimmt hier nicht?

```
void setup() {  
  pinMode(13, OUTPUT);    // Pin als Ausgang setzen  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000)  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Rückblick - Was haben wir letztes Mal gemacht?

Was stimmt hier nicht?

```
void setup() {  
  pinMode(13, OUTPUT);    // Pin als Ausgang setzen  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000)  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Rückblick - Was haben wir letztes Mal gemacht?

Was stimmt hier nicht?

```
void setup() {  
  pinMode(13, OUTPUT);    // Pin als Ausgang setzen  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000)  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Rückblick - Was haben wir letztes Mal gemacht?

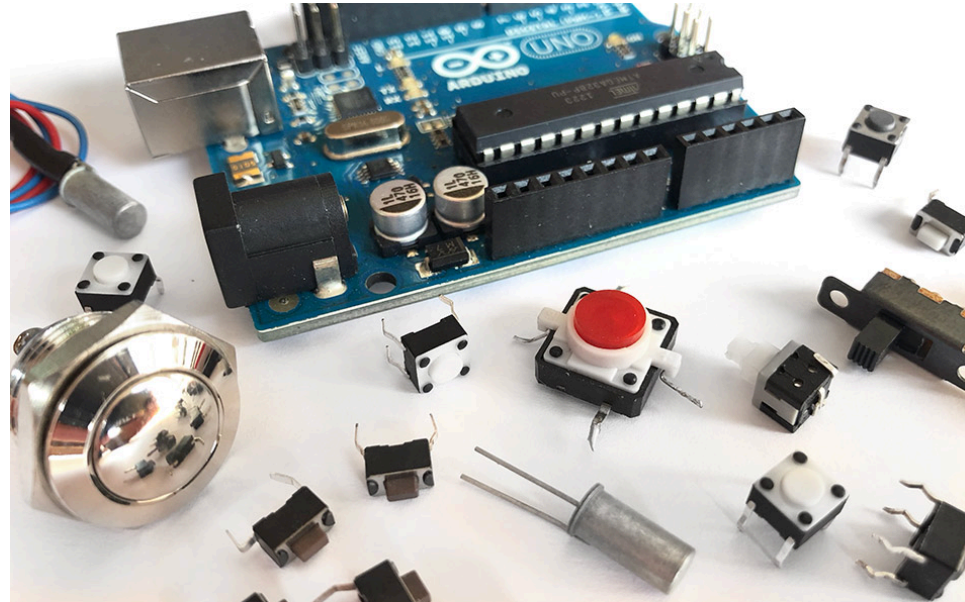
Was stimmt hier nicht?

```
void setup() {  
  pinMode(13, OUTPUT);    // Pin als Ausgang setzen  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000)  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Heute

- Digitale Eingaben lesen
- Entscheidungen treffen mit `if` / `else`
- LED mit Taster steuern

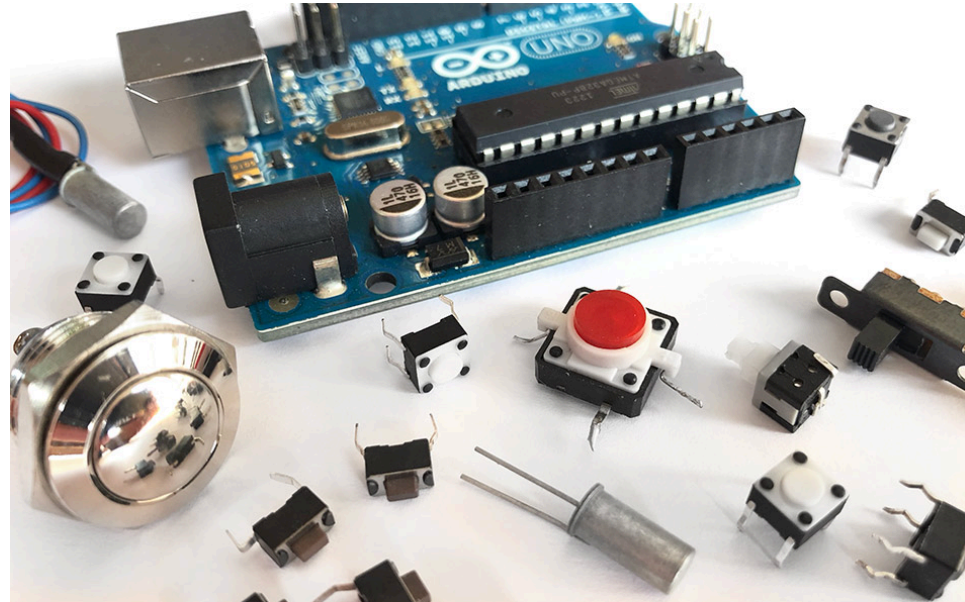
(Heft S. 7-8)



Der Taster

(Heft S. 7-8)

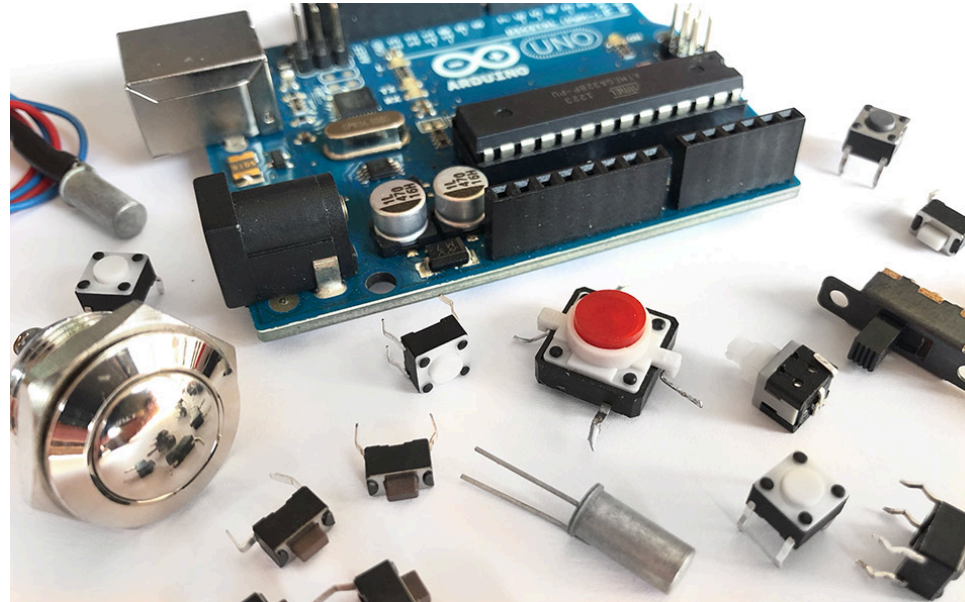
- Digitale Signale haben nur **zwei Zustände**
 - HIGH = 5V
 - LOW = 0V (GND)



Der Taster

(Heft S. 7-8)

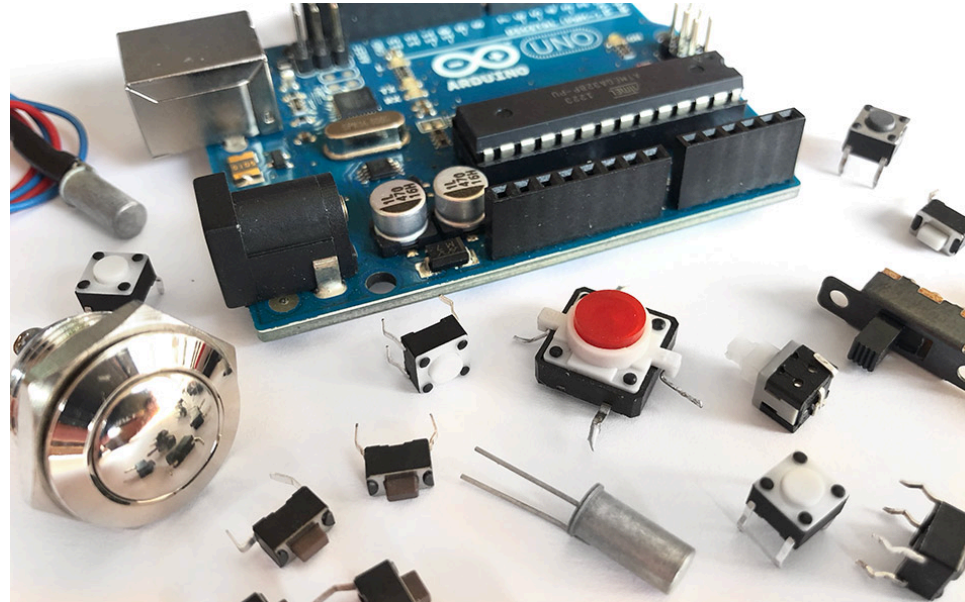
- Digitale Signale haben nur **zwei Zustände**
 - HIGH = 5V
 - LOW = 0V (GND)
- Taster zwischen Pin und GND



Der Taster

(Heft S. 7-8)

- Digitale Signale haben nur **zwei Zustände**
 - HIGH = 5V
 - LOW = 0V (GND)
- Taster zwischen Pin und GND
- Interner Pull-Up Widerstand



pinMode(), digitalRead() und digitalWrite()

```
1 // Aus Termin 1 kennen wir das schon:  
2 pinMode(pin, OUTPUT);  
3 digitalWrite(pin, HIGH); // oder LOW
```

pinMode(), digitalRead() und digitalWrite()

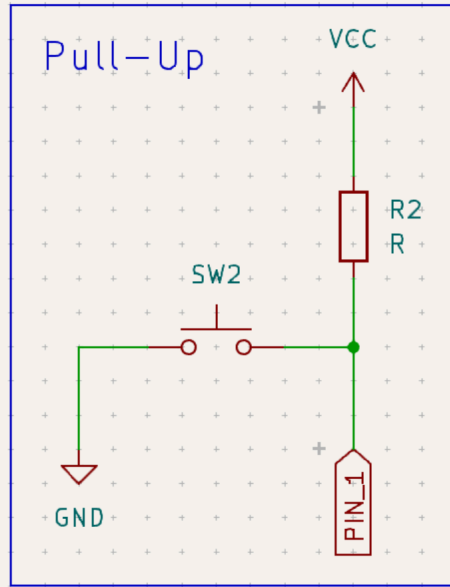
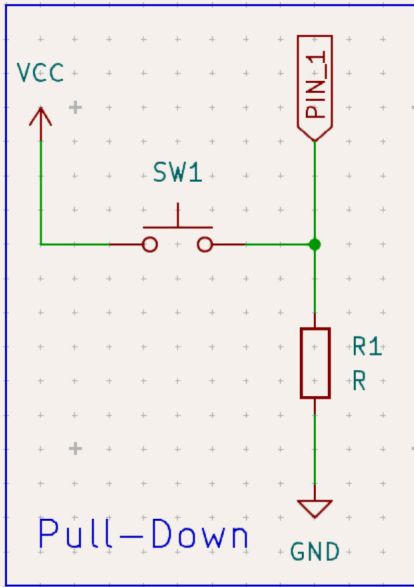
```
1 // Aus Termin 1 kennen wir das schon:
2 pinMode(pin, OUTPUT);
3 digitalWrite(pin, HIGH); // oder LOW
4
5 // Neu heute:
6 pinMode(pin, INPUT_PULLUP); // Pin als Eingang mit Pull-Up
7 digitalRead(pin);           // Liefert HIGH oder LOW zurück
```

pinMode(), digitalRead() und digitalWrite()

```
1 // Aus Termin 1 kennen wir das schon:
2 pinMode(pin, OUTPUT);
3 digitalWrite(pin, HIGH); // oder LOW
4
5 // Neu heute:
6 pinMode(pin, INPUT_PULLUP); // Pin als Eingang mit Pull-Up
7 digitalRead(pin);           // Liefert HIGH oder LOW zurück
```

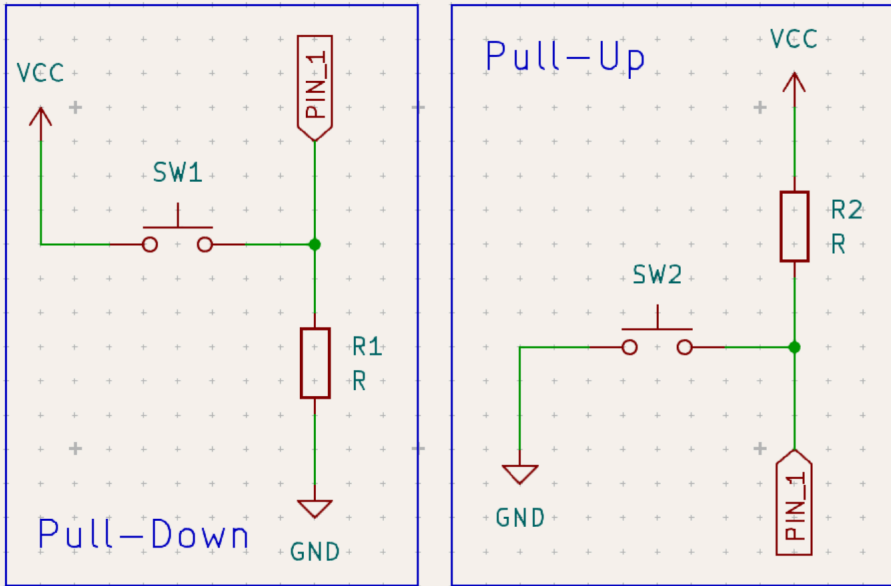
Achtung: Bei INPUT_PULLUP bedeutet gedrückt = **LOW**!

Pull-Up/ Pull-Down Widerstand

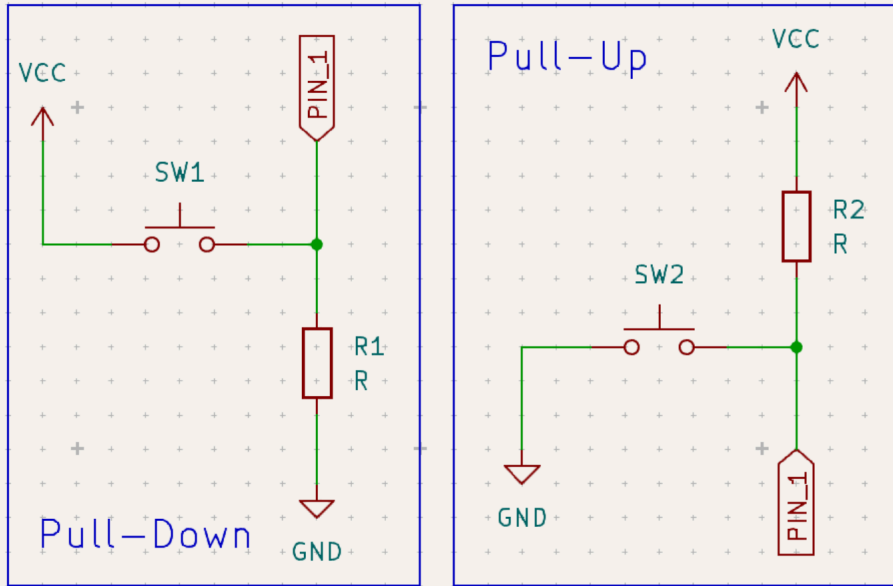


Pull-Up/ Pull-Down Widerstand

- Ein offener Pin "floatet" – er nimmt zufällige Werte an

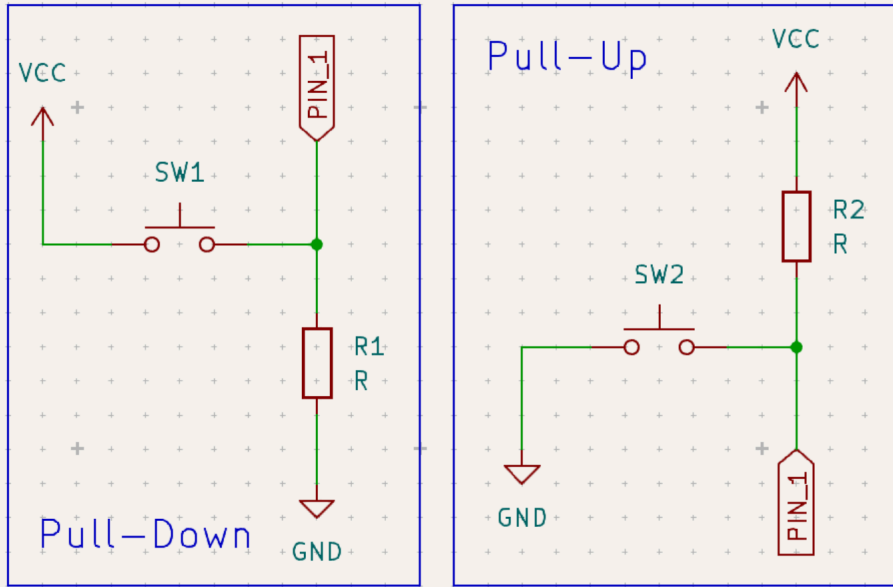


Pull-Up/ Pull-Down Widerstand



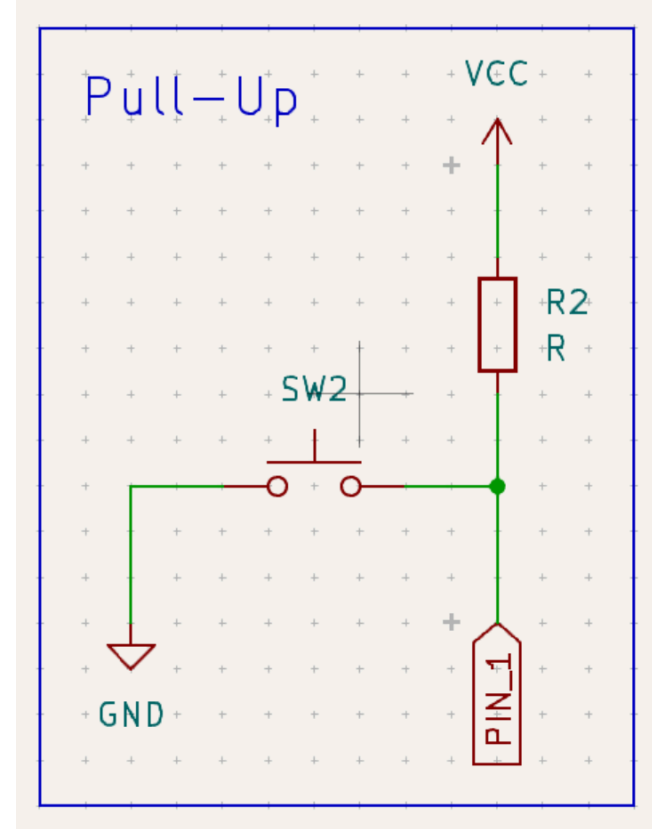
- Ein offener Pin "floatet" – er nimmt zufällige Werte an
- Der Pull-Up Widerstand zieht den Pin auf HIGH, wenn nichts angeschlossen ist

Pull-Up/ Pull-Down Widerstand



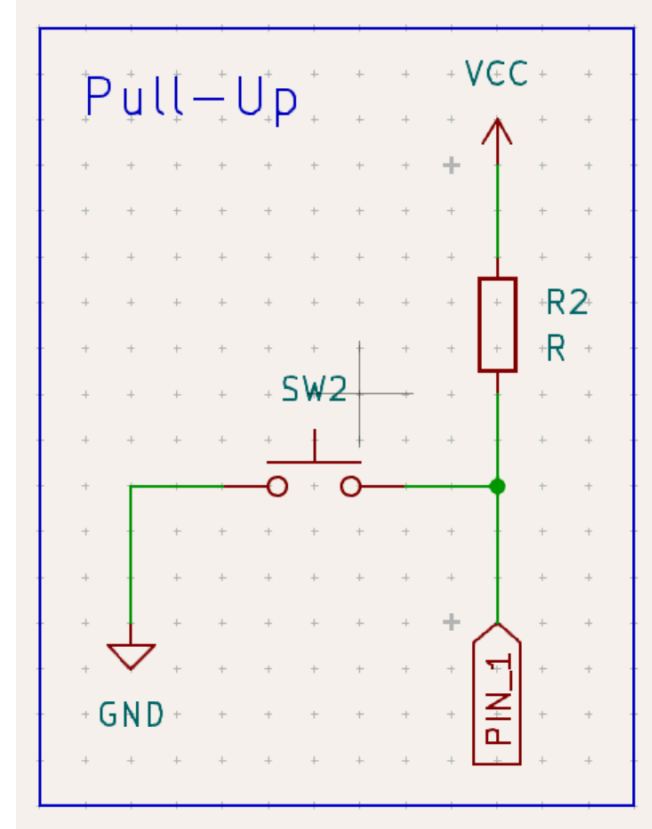
- Ein offener Pin "floatet" – er nimmt zufällige Werte an
- Der Pull-Up Widerstand zieht den Pin auf HIGH, wenn nichts angeschlossen ist
- Der Pull-Down Widerstand zieht den Pin auf LOW, wenn nichts angeschlossen ist

Interne Pull-Up Widerstand



Interner Pull-Up Widerstand

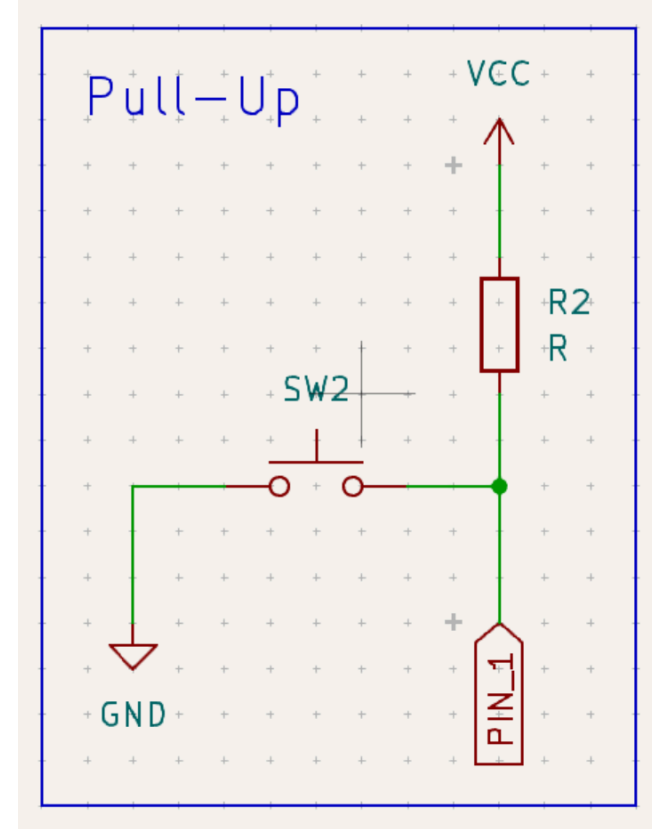
- Widerstand R2 ist im Arduino eingebaut
- Taster zwischen GND und Pin anschließen



Interner Pull-Up Widerstand

- Widerstand R2 ist im Arduino eingebaut
- Taster zwischen GND und Pin anschließen

```
Pull-Up aktivieren: pinMode(13, INPUT_PULLUP);  
Eingang abfragen: digitalRead(13);
```



Bedingungen mit if / else

ve bele

```
if (Bedingung) {  
    // Wird ausgeführt, wenn Bedingung WAHR ist  
} else {  
    // Wird ausgeführt, wenn Bedingung FALSCH ist  
}
```

Bedingungen mit if / else

ve bele

```
if (Bedingung) {  
    // Wird ausgeführt, wenn Bedingung WAHR ist  
} else {  
    // Wird ausgeführt, wenn Bedingung FALSCH ist  
}  
  
if (helligkeit > 500) {  
    digitalWrite(LED_PIN, HIGH);  
} else {  
    digitalWrite(LED_PIN, LOW);  
}
```

Bedingungen mit if / else

ve bele

```
if (Bedingung) {  
    // Wird ausgeführt, wenn Bedingung WAHR ist  
} else {  
    // Wird ausgeführt, wenn Bedingung FALSCH ist  
}
```


Bedingungen mit if / else

ve bele

```
if (Bedingung) {  
    // Wird ausgeführt, wenn Bedingung WAHR ist  
} else {  
    // Wird ausgeführt, wenn Bedingung FALSCH ist  
}
```

Vergleichsoperatoren:

Operator	Bedeutung
==	gleich
!=	ungleich
> / <	größer / kleiner

Häufiger Fehler: `=` vs. `==`

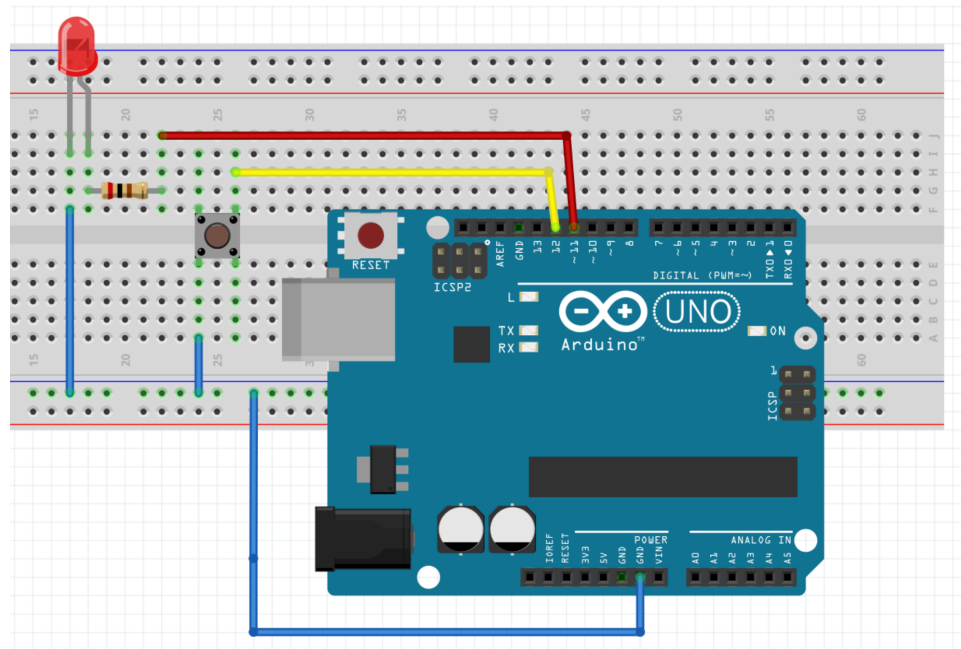
```
int x = 5;    // = ist eine ZUWEISUNG - x bekommt den Wert 5

if (x == 5) {    // == ist ein VERGLEICH - ist x gleich 5?
    // ...
}

if (x = 5) {    // FALSCH! Das ist eine Zuweisung, kein Vergleich!
    // ...
}
```

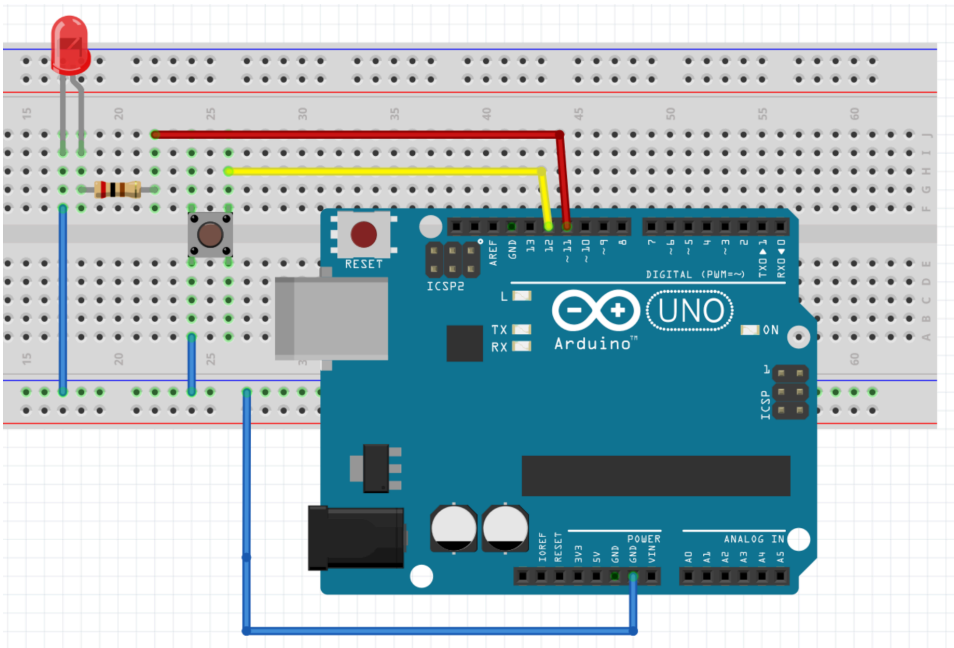
Noch Fragen?

LED mit Taster steuern



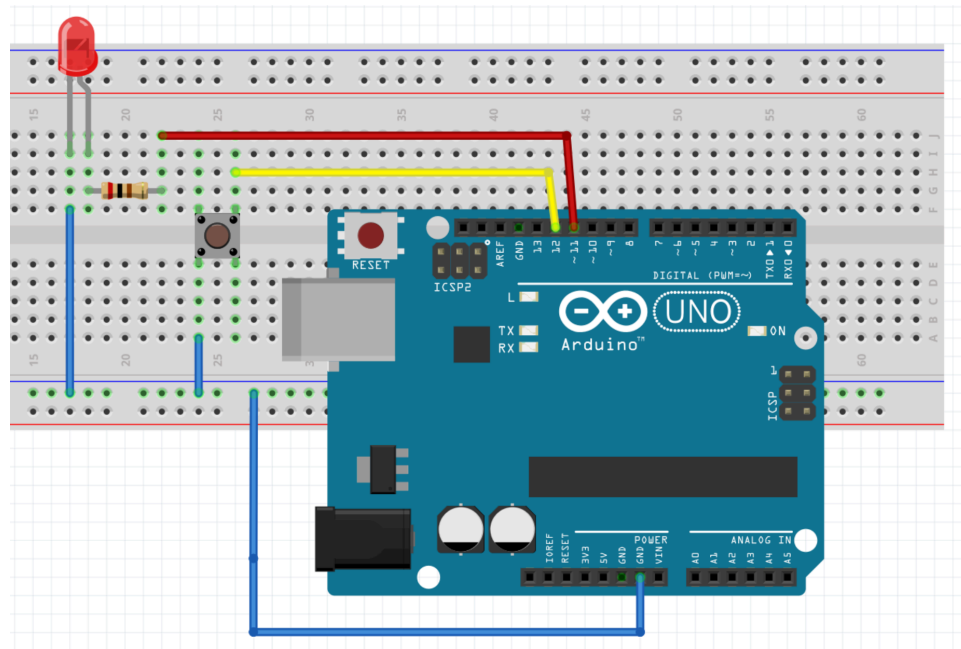
LED mit Taster steuern

- Taster an Pin 12 und GND



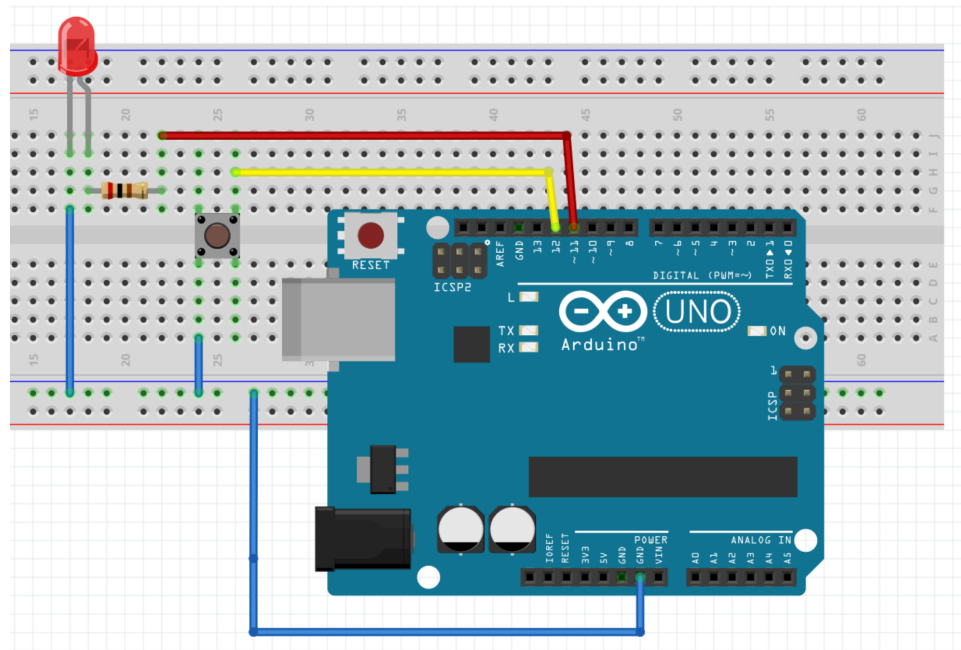
LED mit Taster steuern

- Taster an Pin 12 und GND
- LED + 220Ω Widerstand an Pin 11 und GND



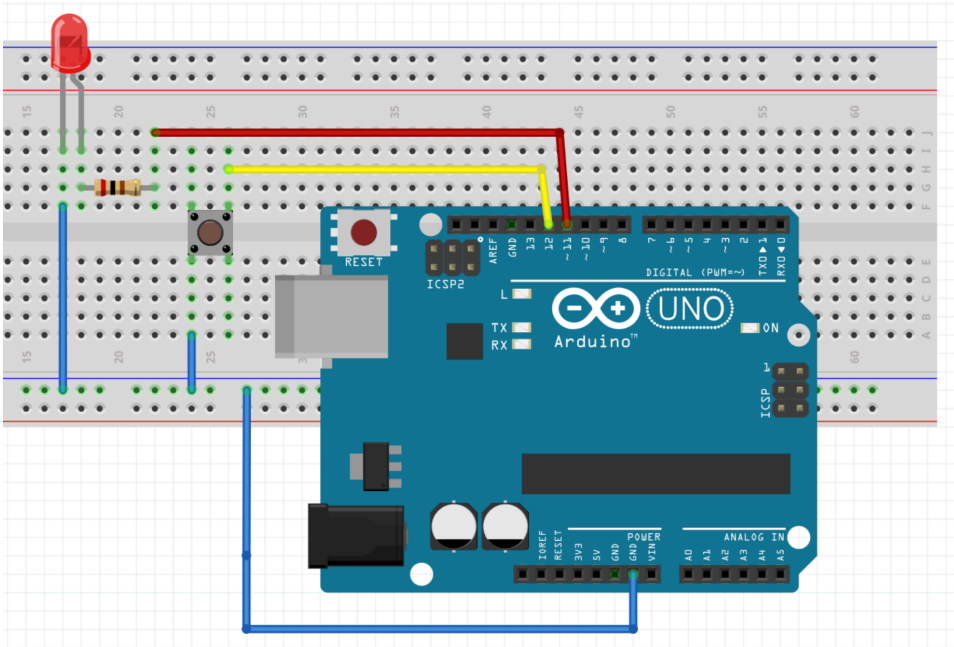
LED mit Taster steuern

- Taster an Pin 12 und GND
- LED + 220Ω Widerstand an Pin 11 und GND
- Solange der Taster gedrückt ist → LED leuchtet



LED mit Taster steuern

- Taster an Pin 12 und GND
- LED + 220Ω Widerstand an Pin 11 und GND
- Solange der Taster gedrückt ist → LED leuchtet
- **Schaltung aufbauen und Code ergänzen**
→ **Los gehts!**



Code-Gerüst: LED mit Taster

```
const int PIN_TASTER = 2;
const int PIN_LED = 9;

void setup() {
  pinMode(PIN_TASTER, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int tasterZustand = digitalRead(PIN_TASTER);

  // TODO: if/else Logik ergänzen
  // Wenn Taster gedrückt (= LOW) -> LED an
  // Sonst -> LED aus

  // TODO: Zustand auf seriellem Monitor ausgeben
}
```

**Warum ist gedrückt = LOW
und nicht HIGH?**

Erweiterung 1 - Toggle-Funktion

Ziel: Bei jedem Tastendruck wechselt die LED ihren Zustand (an → aus → an → aus)

Tipp: Du solltest den Zustand der LED speichern

Erweiterung 1 - Toggle-Funktion - Lösung

```
bool ledZustand = false;    // Zustandsspeicher

void loop() {
    if (digitalRead(PIN_TASTER) == LOW) {
        ledZustand = !ledZustand;    // Zustand umkehren
        digitalWrite(PIN_LED, ledZustand);
        delay(50);    // Einfache Entprellung
    }
}
```

Erweiterung 1 - Toggle-Funktion - Lösung

```
bool ledZustand = false;    // Zustandsspeicher

void loop() {
    if (digitalRead(PIN_TASTER) == LOW) {
        ledZustand = !ledZustand;    // Zustand umkehren
        digitalWrite(PIN_LED, ledZustand);
        delay(50);    // Einfache Entprellung
    }
}
```

Hinweis: Die LED schaltet manchmal mehrmals – das ist das **Prellen** des Tasters!

Erweiterung 2 - Software-Entprellung

Damit ein Tastendruck erkannt wird muss der Taster für mindestens 100ms gedrückt sein!

Erweiterung 2 - Software-Entprellung - Lösungsvorschlag

```
if (digitalRead(PIN_TASTER) == LOW) {  
    delay(100);                // Warte 100ms  
    if (digitalRead(PIN_TASTER) == LOW) { // Immer noch gedrückt?  
        // Echter Tastendruck -> Aktion ausführen  
    }  
}
```

Erweiterung 2 - Software-Entprellung - Lösungsvorschlag

```
if (digitalRead(PIN_TASTER) == LOW) {  
    delay(100);                // Warte 100ms  
    if (digitalRead(PIN_TASTER) == LOW) { // Immer noch gedrückt?  
        // Echter Tastendruck -> Aktion ausführen  
    }  
}
```

Was wäre, wenn du `delay(100)` nicht verwenden kannst, weil der Arduino in dieser Zeit noch andere Dinge tun muss? Das lösen wir im Fortgeschrittenenkurs!

Zusatzaufgabe - Ampelschaltung

Aufbau

- Rote, gelbe und grüne LED
 - Rot => Pin 13
 - Gelb => Pin 12
 - Grün => Pin 11
- Jede LED mit passendem Vorwiderstand
- Zeiten wie bei echter Ampel

Ablauf

Phase	Farbe	Zeit
Stop	Rot	3000ms
Achtung	Rot + Gelb	1000ms
Fahrt	Grün	3000ms
Bremsen	Gelb	1000ms

Zusatzaufgabe - Fußgängerampel

Aufbau

- Zusätzlich rote und grüne LED für Fußgänger
 - Rot => Pin 10
 - Grün => Pin 9
- Taster für Fußgänger
 - Pin 8

Ablauf

Zusätzlich zur normalen Ampel:

- Wenn der Taster gedrückt wurde läuft die Ampel normal weiter, bis das nächste mal rot kommt
- Beide Ampeln müssen für 500ms rot sein
- Fußgängerampel für 3000ms grün
- Beide Ampeln für 500ms rot
- Auto-Ampel läuft bei gelb weiter

Challenge - Türklingel mit Bestätigung

Bei dieser Aufgabe musst du wirklich knobeln - Also lass dich nicht unterkriegen!

Ziel: Du simulierst eine Türklingel mit Türöffner

- Wird Taster 1 (Klingel) gedrückt, blinkt eine rote LED langsam
- Wird Taster 2 (Türöffner) gedrückt, geht die rote LED aus und eine grüne LED blinkt kurz
- Wird Taster 2 nicht innerhalb von ca. 5 Sekunden gedrückt, blinkt die rote LED schneller

Aufgaben:

- Zeichne einen Schaltplan auf Papier
- Baue die Schaltung auf deinem Breadboard auf
- Programmiere die Logik

Zusammenfassung

Digitale Eingabe

- `pinMode(pin, INPUT_PULLUP)`
- `digitalRead(pin)` → HIGH oder LOW

Pull-Up Widerstand

- Pin wird auf HIGH gezogen
- Gedrückt = LOW!

if / else

- `==` zum Vergleichen, nicht `=`
- `else` für den anderen Fall

Taster-Schaltung

- Taster zwischen Pin und GND
- `INPUT_PULLUP` verwenden

Entprellung

- Taster prellen beim Drücken
- `delay(100)` als einfache Lösung